

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 2: ANALYSIS OF RECURSIVE ALGORITHMS

Abdullah Bal, PhD

Objectives



Recap

- Analysis of Loops
- Recursive algorithms



Recursive Algorithm Analysis



Backward substitutions



Recursive- Iterative comparison

Plan for Analysis of Recursive Algorithms

- Decide on a parameter indicating an **input's size**.
- Identify the algorithm's **basic operation**.
- Check whether the number of times the **basic operation** is executed may vary on **different inputs of the same size**.
- If it may, **the worst, average, and best cases** must be investigated **separately**.
- **Set up a recurrence relation** with an appropriate initial condition expressing the number of times the basic operation is executed.
- Solve the recurrence (or, at the very least, establish its solution's order of growth) by **backward substitutions or another method**.

Example 1: Recursive analysis of $n!$

$$n! = 1 \cdot \dots \cdot (n - 1) \cdot n = (n - 1)! \cdot n \quad \text{for } n \geq 1$$

$$0! = 1$$

Recursive definition of $n!$: $F(n) = F(n-1) \cdot n$ for $n \geq 1$

ALGORITHM $F(n)$

//Computes $n!$ recursively

//Input: A nonnegative integer n

//Output: The value of $n!$

if $n = 0$ **return** 1

else return $F(n - 1) * n$

Solving the recurrence for $M(n)$

- To determine a solution uniquely, we need an **initial condition** that tells us the value with which the sequence starts.
- We can obtain this value by inspecting the condition that makes the algorithm **stop its recursive calls**:

if $n = 0$ return 1.

- We are dealing here with **two recursively** defined functions.
- The first is the **factorial function $F(n)$** itself; it is defined by the recurrence

$F(n) = F(n-1) \cdot n$ for every $n > 0$,

$F(0) = 1$.

- The second is the number of **multiplications $M(n)$** needed to compute $F(n)$ by the recursive algorithm.

Backward Substitutions

- It will be more useful to arrive at it in a **systematic fashion**.
- We use what can be called the method of **backward substitutions**.
- The method's idea (and the reason for the name) is immediately clear from the way it applies to solving our particular recurrence:

$$\begin{aligned}M(n) &= M(n - 1) + 1 \\&= [M(n-2)+1]+1=M(n-2)+2 \\&= [M(n - 3) + 1] + 2 = M(n - 3) + 3.\end{aligned}$$

$$\text{substitute } M(n - 1) = M(n - 2) + 1$$

$$\text{substitute } M(n-2)=M(n-3)+1$$

- After inspecting the first three lines, we see an **emerging pattern**, which makes it possible to predict not only the next line (what would it be?) but also a general formula for the pattern:

$$M(n) = M(n - i) + i$$

Solving the recurrence for $M(n)$

- Let us take advantage of the **initial condition** given.
- Since it is specified for $n = 0$, we have to substitute $i = n$ in the pattern's formula to get the ultimate result of our backward substitutions:

$$M(n) = M(n - 1) + 1 = \dots = M(n - i) + i = \dots = M(n - n) + n = n$$

- The **simple iterative algorithm** that accumulates the **product of n** consecutive integers requires the same number of multiplications.

Example 2: Backward Substitutions

Problem: What is the time complexity of the following recursive algorithm?

$$T(n) = T(n - 1) + n$$

$$T(1)=1$$

Example 2: Backward Substitutions

Solution: Guess the pattern (start with given recurrence)

$$T(n) = T(n - 1) + n$$

substitute $T(n - 1) = T(n - 2) + (n - 1)$

$$T(n) = T(n - 2) + (n - 1) + n$$

substitute $T(n - 2) = T(n - 3) + (n - 2)$

$$T(n) = T(n - 3) + (n - 2) + (n - 1) + n$$

....

$$T(n) = T(n - i) + (n - i + 1) + (n - i + 2) + \dots + n$$

$$n - i = 1 \rightarrow i = n - 1$$

$$T(n) = T(1) + 2 + 3 + \dots + n$$

$$T(n) = 1 + 2 + 3 + \dots + n = n(n + 1)/2$$

Time complexity: $\Theta(n^2)$

Example 3: Backward Substitutions

Problem: What is the time complexity of the following recursive algorithm?

$$T(n) = n * T(n - 1)$$

$$T(1)=1$$

Example 3: Backward Substitutions

Solution: Guess the pattern (start with given recurrence)

$$T(n) = n * T(n - 1)$$

substitute $T(n - 1) = (n-1) * T(n - 2)$

$$T(n) = n * (n-1) * T(n-2)$$

substitute $T(n - 2) = (n-2) * T(n - 3)$

$$T(n) = n * (n-1) * (n-2) * T(n-3)$$

....

$$T(n) = n * (n-1) * (n-2) * * (n-i+1) * T(n-i)$$

$$n-i=1 \rightarrow i=n-1$$

$$T(n) = T(1) * 2 * 3 * * (n-1) * n$$

$$T(n) = n!$$

Time complexity: $\Theta(n!)$

Example 4: Backward Substitutions

Problem: What is the time complexity of the following recursive algorithm?

$$T(n) = T(n/3) + 1$$

$$T(1)=1$$

Example 4: Backward Substitutions

Solution: Guess the pattern (start with given recurrence)

$$T(n) = T(n/3) + 1$$

substitute $T(n/3) = T(n/3^2) + 1$

$$T(n) = T(n/3^2) + 1 + 1 = T(n/3^2) + 2$$

substitute $T(n/3^2) = T(n/3^3) + 1$

$$T(n) = T(n/3^3) + 1 + 2 = T(n/3^3) + 3$$

....

$$T(n) = T(n/3^i) + i$$

$$n/3^i = 1 \rightarrow i = \log_3 n$$

$$T(n) = 1 + \log_3 n$$

Time complexity: $\Theta(\log_3 n)$

Example 5: Backward Substitutions

Problem: What is the time complexity of the following recursive algorithm?

$$T(n) = 2T(n - 1) + 1$$

$$T(1)=0$$

Example 5: Backward Substitutions

Solution: Guess the pattern (start with given recurrence)

$$T(n) = 2T(n-1) + 1$$

substitute $T(n-1) = 2T(n-2) + 1$

$$T(n) = 2(2T(n-2) + 1) + 1 = 2^2 T(n-2) + 2 + 1$$

substitute $T(n-2) = 2T(n-3) + 1$

$$T(n) = 2^2(2T(n-3) + 1) + 2 + 1 = 2^3 T(n-3) + 2^2 + 2 + 1$$

....

$$T(n) = 2^i T(n-i) + 2^{(i-1)} + \dots + 2^2 + 2 + 1$$

$$T(n) = 2^i T(n-i) + 2^i - 1$$

$$n-i=1 \rightarrow i=n-1$$

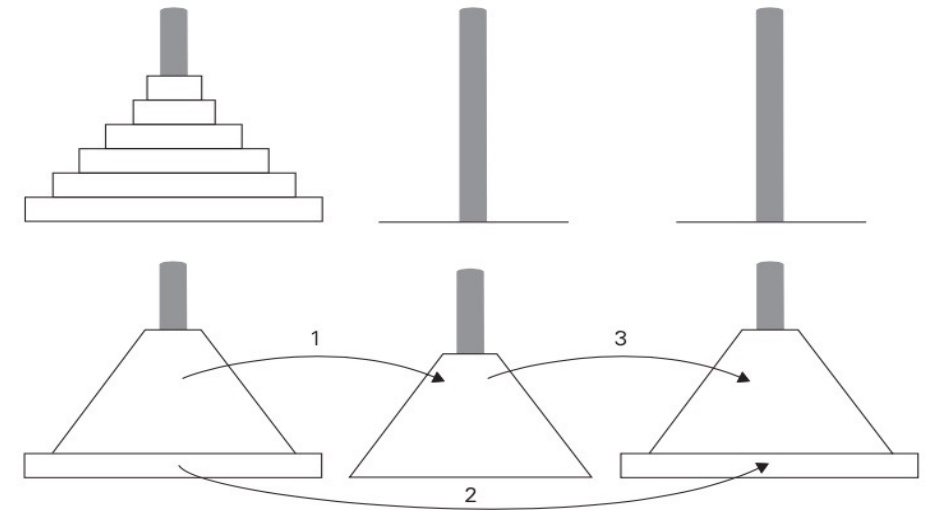
$$T(n) = 2^{n-1} T(1) + 2^{n-1} - 1 = \frac{2^n}{2} - 1$$

Time complexity: $\Theta(2^n)$

$$\begin{aligned} s_n &= ar^0 + ar^1 + \dots + ar^{n-1} \\ &= \sum_{k=0}^{n-1} ar^k = \sum_{k=1}^n ar^{k-1} \\ &= \begin{cases} a \left(\frac{1-r^n}{1-r} \right) & r \neq 1 \\ an & \text{otherwise} \end{cases} \end{aligned}$$

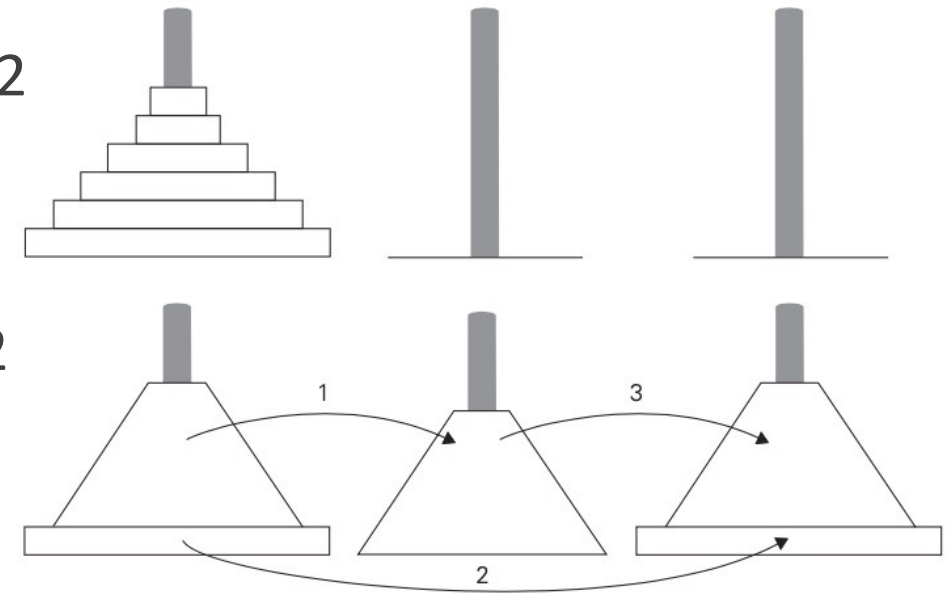
Example 6: The Tower of Hanoi Puzzle

- We have n disks of different sizes that can slide onto any of three pegs.
- Initially, all the disks are on the first peg in order of size, the largest on the bottom and the smallest on top.
- The goal is to move all the disks to the third peg, using the second one as an auxiliary, if necessary.
- We can move only one disk at a time, and it is forbidden to place a larger disk on top of a smaller one.



Example 6: The Tower of Hanoi Puzzle (Cont.)

- The problem has an elegant **recursive solution**.
- To move $n > 1$ disks from peg 1 to peg 3 (with peg 2 as auxiliary), we **first move recursively $n - 1$ disks** from peg 1 to peg 2 (with peg 3 as auxiliary), then move the largest disk directly from peg 1 to peg 3, and, finally, move recursively $n - 1$ disks from peg 2 to peg 3 (using peg 1 as auxiliary).
- Of course, **if $n = 1$, we simply move the single disk** directly from the source peg to the destination peg.



Example 6: The Tower of Hanoi Puzzle (Cont.)

- Apply the general plan outlined above to the Tower of Hanoi problem.
- The number of disks n is the obvious choice for the **input's size** indicator, and so is **moving one disk** as the algorithm's **basic operation**.
- Clearly, the number of moves $M(n)$ depends on n only, and we get the following recurrence equation for it:

$$M(n) = M(n-1) + 1 + M(n-1) \quad \text{for } n > 1$$

- With the obvious initial condition $M(1) = 1$, we have the following recurrence relation for the number of moves $M(n)$:

$$M(n) = 2M(n-1) + 1 \quad \text{for } n > 1$$

$$M(1) = 1.$$

Example 6: The Tower of Hanoi Puzzle (Cont.)

- We solve this recurrence by the same method of backward substitutions:

$$M(n) = 2M(n - 1) + 1 \qquad \text{sub. } M(n - 1) = 2M(n - 2) + 1$$

$$= 2[2M(n-2)+1]+1 = 2^2M(n-2)+2+1 \quad \text{sub. } M(n-2)=2M(n-3)+1$$

$$= 2^2[2M(n-3)+1]+2+1 = 2^3M(n-3)+2^2+2+1.$$

- The pattern of the first three sums on the left suggests that the next one will be

- $2^4M(n - 4) + 2^3 + 2^2 + 2 + 1,$

- Generally, after i substitutions, we get

$$M(n) = 2^iM(n - i) + 2^{i-1} + 2^{i-2} + \dots + 2 + 1$$

Example 6: The Tower of Hanoi Puzzle (Cont.)

$$\begin{aligned} s_n &= ar^0 + ar^1 + \dots + ar^{n-1} \\ &= \sum_{k=0}^{n-1} ar^k = \sum_{k=1}^n ar^{k-1} \\ &= \begin{cases} a \left(\frac{1-r^n}{1-r} \right) & r \neq 1 \\ an & \text{otherwise} \end{cases} \end{aligned}$$

$$M(n) = 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \dots + 2 + 1 = 2^i M(n-i) + 2^i - 1$$

- Since the initial condition is specified for $n = 1$, which is achieved for $i = n - 1$,
- We get the following formula for the solution to recurrence:

$$M(n) = 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 = 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$$

Example 6: The Tower of Hanoi Puzzle (Cont.)

- We get the following formula for the solution to recurrence:

$$M(n) = 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 = 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1.$$

- We have an exponential algorithm $\Theta(2^n)$, which will run for an unimaginably long time even for moderate values of n .

Example 7: Fibonacci numbers

- The Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

- The Fibonacci recurrence:

$$F(n) = F(n-1) + F(n-2) \text{ for } n > 1$$

- Two initial conditions

$$F(0) = 0, \quad F(1) = 1$$

Example 7: Fibonacci numbers recursive

ALGORITHM $F(n)$

//Computes the n th Fibonacci number recursively by using its definition

//Input: A nonnegative integer n

//Output: The n th Fibonacci number

if $n \leq 1$ **return** n

else return $F(n - 1) + F(n - 2)$

- The algorithm's **basic operation** is clearly **addition**, so let $A(n)$ be the number of additions performed by the algorithm in computing $F(n)$.
- The numbers of additions needed for computing $F(n - 1)$ and $F(n - 2)$ are $A(n - 1)$ and $A(n - 2)$, respectively.
- The algorithm needs one more addition to compute their sum.

Example 7: Fibonacci numbers recursive(Cont.)

- Thus, we get the following recurrence for $A(n)$:

$$A(n) = A(n-1) + A(n-2) + 1 \quad \text{for } n > 1$$

$$A(0) = 0, \quad A(1) = 0$$

- The recurrence

$A(n) - A(n-1) - A(n-2) = 1$ is quite similar to recurrence

$$F(n) - F(n-1) - F(n-2) = 0,$$

- But its right-hand side is not equal to zero.
- Such recurrences are called *inhomogeneous*.
- There are general techniques for solving inhomogeneous recurrences (discrete mathematics),
- For this particular recurrence, a special trick leads to a faster solution.

Example 7: Fibonacci numbers recursive(Cont.)

- We can reduce our inhomogeneous recurrence to a homogeneous one by rewriting it as $[A(n)+1]-[A(n-1)+1]-[A(n-2)+1]=0$ and substituting $B(n) = A(n) + 1$
 $B(n)-B(n-1)-B(n-2)=0,$
 $B(0) = 1, B(1) = 1.$
- This homogeneous recurrence can be solved.
- But it can actually be avoided by noting that $B(n)$ is, in fact, the same recurrence as $F(n)$ except that it starts with two 1's and thus runs one step ahead of $F(n)$.
 $B(n) = F(n + 1)$

Example 7: Fibonacci numbers recursive(Cont.)

$$A(n) = B(n) - 1 = F(n + 1) - 1 = \frac{1}{\sqrt{5}}(\phi^{n+1} - \hat{\phi}^{n+1}) - 1.$$

$$B(n) = F(n + 1)$$

Hint:

$$F(n) = \frac{1}{\sqrt{5}}(\phi^n - \hat{\phi}^n),$$

where $\phi = (1 + \sqrt{5})/2 \approx 1.61803$ and $\hat{\phi} = -1/\phi \approx -0.61803$

- Hence, $A(n) \in \Theta(\phi^n)$,

Example 8: Fibonacci numbers (Iterative)

- We can obtain a much faster algorithm by simply computing the successive elements of the Fibonacci sequence **iteratively**.
- This algorithm clearly makes $n - 1$ additions. Hence, it is linear as a function of **n** .
- Time complexity **$\Theta(n)$**

ALGORITHM *Fib*(n)

//Computes the n th Fibonacci number iteratively |

//Input: A nonnegative integer n

//Output: The n th Fibonacci number

$F[0] \leftarrow 0; F[1] \leftarrow 1$

for $i \leftarrow 2$ **to** n **do**

$F[i] \leftarrow F[i - 1] + F[i - 2]$

return $F[n]$

Summary

- Recursive algorithm analysis
- Backward Substitution
- Recursive-Iterative comparison

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.